Tkinter UI Framework

This document explains how the kiosk's graphical interface is built with Tkinter, the design patterns used, and how the component system works.

Why Tkinter

Tkinter is Python's built-in GUI toolkit. It was chosen for this project because:

- It ships with Python on all platforms, requiring no additional installation.
- It runs natively as a desktop application without needing a web browser or heavy desktop environment.
- It supports touchscreen input out of the box.
- It produces lightweight applications suitable for low-spec kiosk hardware.

Screen Management Pattern

The application uses a single-window, screen-stack architecture. All screens are `tk.Frame` subclasses created once at startup and layered on top of each other using `place(x=0, y=0, relwidth=1, relheight=1)`:

```
self._screens = {
   "tap":             TapScreen(self.root, self),
   "signin":          SignInScreen(self.root, self),
   "signature":       SignatureScreen(self.root, self),
   # ...
}

for screen in self._screens.values():
   screen.place(x=0, y=0, relwidth=1, relheight=1)
```

Navigation lifts the target screen to the top of the stack:

```
def show_screen(self, name, **kwargs):
   screen = self._screens[name]
   screen.lift()
   if hasattr(screen, "on_show"):
        screen.on_show(**kwargs)
```

Each screen implements an `on_show(**kwargs)` method that refreshes its data and resets its state when it becomes visible. This avoids stale data and ensures a clean state on every visit.

This pattern was chosen over dynamically creating and destroying frames because: - Instantiation is expensive (building widget trees). - All screens share the same parent window, making layout consistent. - Data is refreshed via `on_show()` rather than reconstruction.

Layout Techniques

Absolute Positioning with `place()`

Most screens use `place()` for pixel-precise layout within a fixed 1280x720 design canvas. This was chosen because the target hardware (the Windows kiosk touchscreen) has a known, fixed screen size, so responsive layout adds complexity without benefit.

```
self._search_entry.place(x=60, y=100, width=584, height=44)
```

Geometry Managers

Tkinter provides three geometry managers, and the project uses all three in different contexts:

- **`place()`** -- Used for the main screen layout (absolute coordinates within the 1280x720 frame) and for overlay elements like dropdowns.
- **`pack()`** -- Used inside scrollable containers and form layouts where widgets stack vertically.
- **`grid()`** -- Not used directly in this project; `pack()` with `side="left"` handles horizontal arrangements.

Split Panel Layout

Several screens (SignIn, Signature, RunSheets) use a two-panel layout created with `place()`:

```
left = tk.Frame(self, bg=CREAM)
left.place(x=0, y=0, width=704, height=720)

right = tk.Frame(self, bg=CREAM)
right.place(x=704, y=0, width=576, height=720)
```

Reusable Widgets

The `components/widgets.py` module provides custom widget classes that are used throughout the kiosk interface.

TouchButton

A `tk.Button` subclass designed for touchscreen use. It adds: - Hover colour transitions (mouse enter/leave events change the background). - Configurable colours, fonts, and sizes through constructor parameters. - Flat styling with no border or relief.

```
class TouchButton(tk.Button):
    def __init__(self, parent, text, command=None,
                 bg=None, fg=None, hover_bg=None, font_cfg=None, ...):
```

StyledEntry

A `tk.Entry` subclass that applies the theme's entry styling automatically. Provides `get_value()` and `set_value()` convenience methods.

DayPicker

A row of Mon-Fri toggle buttons. Each button is a `tk.Button` that toggles a `BooleanVar`. State is stored and retrieved as a comma-separated string (e.g. `"Mon,Tue,Wed"`).

This widget demonstrates the observer pattern -- each button's command callback toggles the variable and refreshes the button's appearance.

SignatureCanvas

A `tk.Canvas` subclass that captures freehand drawing input: - Records stroke coordinates as `(x1, y1, x2, y2)` tuples in a list. - Draws lines in real-time using `create_line()`. - Exports stroke data as a serialised byte string for storage in the database. - Uses `grab_set()` and `grab_release()` to prevent touch events from propagating during drawing. - Exposes `has_signature()` (returns `True` once at least one stroke has been drawn), which `SignatureScreen._submit()` checks before recording a sign event — submitting with an empty canvas is blocked with a popup asking the user to sign first.

```
def _draw(self, e):
    self.create_line(self._last_x, self._last_y, e.x, e.y,
                     fill="black", width=2, smooth=True, capstyle="round")
    self._strokes.append((self._last_x, self._last_y, e.x, e.y))
```

ScrollFrame

A scrollable container built from a `tk.Canvas` with a `tk.Scrollbar`. Content is placed inside `scroll.inner` (a `tk.Frame` window embedded in the canvas).

Supports three scroll methods: - **Mouse wheel** (desktop) - **Touch drag** (touchscreen) -- captures `ButtonPress-1` and `B1-Motion` to calculate scroll delta - **Scrollbar** (fallback)

```
def _ts_move(self, e):
    delta = self._ty0 - e.y_root
    self._canvas.yview_scroll(int(delta / 8), "units")
    self._ty0 = e.y_root
```

PopupOverlay

A `tk.Toplevel` window used for confirmation dialogs and status messages. Features: - Centred on screen with `overrideredirect(True)` (no window decorations). - Modal via `grab_set()` (blocks interaction with the main window). - Auto-sizing based on message length. - Colour-coded accent bar (red for errors, green for success). - Scrollable message text for long content.

PINPad

A numeric keypad widget for PIN entry. Displays a masked entry field and a 4x3 grid of buttons (0-9, DEL, ENT). Callbacks are fired through an `on_confirm` parameter.

Dropdown Pattern

The sign-in and run sheet screens implement custom dropdown lists as overlay frames:

1. A `tk.Frame` (the dropdown) is positioned directly below the search entry using `place()`.
2. It is shown on `FocusIn` and hidden on `FocusOut` (with a short delay to allow click events to fire first).
3. The dropdown is populated by querying the database and dynamically creating label widgets for each result.
4. Hover effects are added by binding `<Enter>` and `<Leave>` events to change background colours.
5. The dropdown is lifted above other widgets with `.lift()`.

This approach was used instead of Tkinter's built-in `OptionMenu` or `Combobox` because those widgets are not designed for touch interaction and cannot display scrollable lists of dynamic data.

Event-Driven Programming

Tkinter is fundamentally event-driven. The application registers callbacks for user actions:

- **Button clicks**: `command=` parameter or `bind("<ButtonPress-1>", callback)`.
- **Text changes**: `StringVar.trace_add("write", callback)` with debouncing via `after()`.
- **Focus changes**: `bind("<FocusIn>", ...)` and `bind("<FocusOut>", ...)` for dropdown show/hide.
- **Key presses**: `bind("<Return>", ...)` for form submission, `bind("<Control-q>", ...)` for quit.

Debouncing

Search inputs use debouncing to avoid querying the database on every keystroke:

```
def _on_search_change(self, *_):
    if self._search_after:
        self.after_cancel(self._search_after)
    self._search_after = self.after(100, self._do_search)
```

This cancels any pending search and schedules a new one 100ms in the future. The search only executes once the user stops typing.

Theme System

All colours, fonts, and styling are centralised in `theme.py`. This provides:

- A single source of truth for the visual design.
- Consistent styling across all screens.
- Easy theme changes without modifying individual screen files.

The theme defines colour constants (page background, content background, button colours, text colours), font tuples, and helper functions like `style_entry()` that apply a standard look to entry widgets.

Virtual Keyboard

Two keyboard implementations exist:

VirtualKeyboard (virtual_keyboard.py)

Used on Windows. A full QWERTY keyboard that appears as a `tk.Toplevel` window anchored to the bottom of the screen. Features: - Shift key support with auto-release after one character. - Normal and shifted key layouts. - Close button to dismiss. - Positioned and sized to match the screen width.

The `App` class manages keyboard visibility by listening to `FocusIn` and `FocusOut` events on all `Entry` widgets. A short delay on `FocusOut` prevents the keyboard from flickering when focus moves between entries.

OnScreenKeyboard (widgets.py)

An embedded keyboard widget with QWERTY and numeric modes. This is a `tk.Frame` that can be placed inside any container. It is not currently used in the main application but is available as a component.

System Keyboard (Linux)

On Linux, the application does not use either built-in keyboard. Instead, it relies on the system's on-screen keyboard (e.g. Squeekboard, Onboard) which is managed by the window manager. The kiosk window is set to a fixed 1280x720 size (rather than fullscreen) so the window manager can resize it when the system keyboard appears.